# Load-Aware Flow Placement for Synchronised Ring-AllReduce Traffic in a Spine–Leaf Fabric


## Group 9[1]

[1]**CS5229 Advanced Computer Networks, National University of Singapore**



## ABSTRACT

Even with ECMP, multiple-1D-ring AllReduce on a spine–leaf fabric runs far above its ideal completion time (139.4 ms against 76.8 ms at 32 hosts and 32 MB). We trace this to hash collisions among the few large, synchronised flows each ToR sends per ring step: a step ends only when its slowest flow does, and flows sharing an uplink with more than their fair share of peers take up to $3\times$ longer. We replace ECMP's oblivious hash with *load-aware flow placement*: at its first packet, each flow goes to the equal-cost port that maximises its expected capacity share $C_p/(n_p+1)$, and is then pinned there, so packets are never reordered. On the public microbenchmarks (32/64/128 hosts, 32–128 MB, 10 seeds), completion time falls by 26.7% on average (18–32% per cell). The scheme also holds under a mid-run link failure, asymmetric and mixed-speed uplinks, 2 or 6 paths, and two tenants ($-15\%$ to $-44\%$).



Keywords: ECMP, load balancing, collective communication, RDMA, ring AllReduce


## PROBLEM FORMULATION

**Workload and ideal.** Each ring holds one host per pod, so pod $i$ sends all its ring traffic to pod $i{+}1$. With $H$ hosts per ToR, a ToR launches $H$ inter-pod flows per step over its $P{=}4$ uplinks, which are oversubscribed 2:1, 4:1 and 8:1 at 32, 64 and 128 hosts. With perfect balancing each flow gets $4C/H$ (5, 2.5 and 12.5 Gb/s), so a four-host ring AllReduce of $S$ bytes ideally takes $6\cdot(S/4)/(4C/H)$, e.g. 153.6 ms at 32 hosts and 64 MB.

**Observation.** With the reference ECMP, the 32-host, 32 MB job takes 139.4 ms ($1.81\times$ ideal). We recomputed the reference ECMP hash (murmur3 over the 5-tuple, seed `0xdeadbeef`) for every flow in `fct.txt` and counted, at each flow's start, the flows from the same ToR active on the same uplink. Mean FCT rises from 11.9 ms for a flow alone on its uplink to 30–38 ms when it shares the uplink with 3–7 others (Fig. 1a), and 24% of flows start on an uplink carrying 4 or more flows, against a fair share of 2.

**Why ECMP collides.** (i) 8 flows over 4 uplinks is a balls-into-bins draw Raab and Steger (1998): under uniform hashing the expected maximum is 3.54 flows against a mean of 2 (Monte Carlo), a $1.77\times$ hot-link slowdown. (ii) Every step opens a new QP with a new source port, so each step is a fresh draw. (iii) A ring step is a barrier, so the job waits for its unluckiest flow. The model predicts that ECMP improves with scale ($1.53\times$ at 16 flows per ToR, $1.37\times$ at 32); measured ECMP JCT goes from $1.77\times$ to $1.70\times$ to $1.42\times$ ideal (Table 1). Low-entropy AI traffic defeats ECMP in production RoCE fabrics too Gangidi et al. (2024).

## SOLUTION DESIGN: LOAD-AWARE FLOW PLACEMENT

The scheme is `LB_MODE 2` (`load-balancing-customized.{h,cc}`), plus one line in `switch-node.cc` that lets the module read its switch's port rates and egress queue bytes. CC, ECN and queueing stay at their defaults.

**Placement and pinning.** At any switch with more than one equal-cost next hop, the first packet of an RDMA data flow (5-tuple plus priority) picks the port $p$ in `m_dstIPRouting[dip]` that is lexicographically best by: $(n_p+1)/C_p$, the inverse of the share the flow expects after joining ($n_p$ = active pinned flows, $C_p$ = link rate); then the fewest flows on $p$ to the same destination ToR, a proxy for the

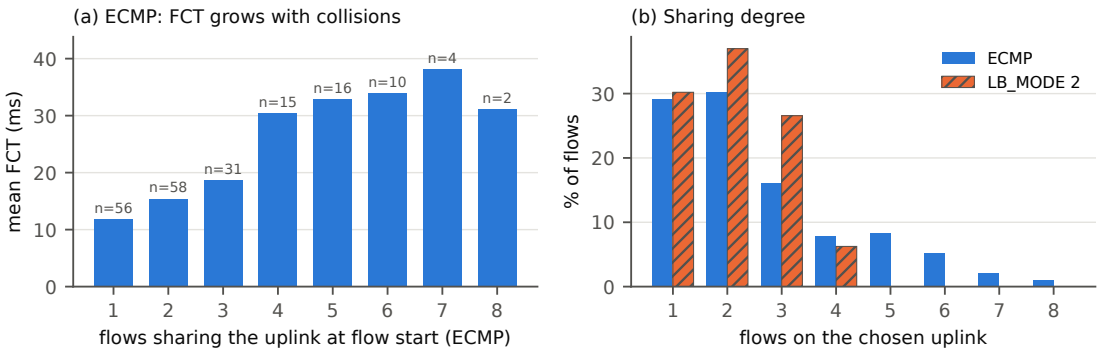



**Figure 1.** 32 hosts / 32 MB / seed 1. (a) ECMP: mean FCT by number of flows on the flow's uplink at its start. (b) Distribution of that sharing degree, ECMP (recomputed hash) vs. LB_MODE 2 (placement log); fair share is 2.

spine's downlink; then the smallest egress queue; then a hash-rotated port order. Every later packet of the flow uses the same port, so there is no reordering and no go-back-N penalty. Control packets and single-path destinations use the reference 5-tuple hash.

**Flow lifetime.** RDMA QPs carry no FIN, so a flow idle for more than 500 µs is treated as finished and its counters are released. That is longer than the packet gap at DCQCN's minimum rate (80 µs) and than a ToR queue drain, so a flow re-placed after going idle has nothing in flight on its old path.

**Robustness by construction.** The path count is always `nexthops.size()`. The pinned port *value* (not an index) is checked against the live routing table on every packet, so a `LINK_DOWN` that removes it triggers re-placement instead of a stale index. Counters are keyed by port value. Nothing assumes the ring structure; traffic outside the collective is just more flows.

**Prior work.** ECMP Hopps (2000) is load-oblivious. CONGA Alizadeh et al. (2014) needs fabric-wide feedback. Flowlets Vanini et al. (2017) need idle gaps that paced RDMA flows rarely have, and packet spraying Ghorbani et al. (2017) reorders packets, which go-back-N punishes. We adapt the idea behind Hedera's placement of large flows Al-Fares et al. (2010), but make it local, online and per-switch.

## EVALUATION

**Setup.** ASTRA-sim 2.0 over NS-3 Won et al. (2023), the shipped spine–leaf configs, DCQCN Zhu et al. (2015), RoCE go-back-N. Within each comparison only `LB_MODE` differs (1 = reference ECMP, 2 = ours). JCT is the latest per-host finish time, averaged over 10 seeds. Part of the sweep used an optimised build of the same source, verified to give byte-identical results.

| Hosts (link) | 32 MB | 64 MB | 128 MB |
|---|---|---|---|
| 32 (10G) | 135.6 → 98.1 (−27.7%) | 270.2 → 191.2 (−29.2%) | 545.3 → 375.1 (−31.2%) |
| 64 (10G) | 261.1 → 177.8 (−31.9%) | 522.4 → 358.0 (−31.5%) | 1042.0 → 708.7 (−32.0%) |
| 128 (100G) | 43.8 → 35.7 (−18.4%) | 86.9 → 70.1 (−19.3%) | 173.2 → 139.8 (−19.3%) |

**Table 1.** Microbenchmarks: mean JCT in ms, ECMP → LB_MODE 2 (change), 10 seeds. Ideal: 76.8/153.6/307.2 (32 hosts), 153.6/307.2/614.4 (64), 30.7/61.4/122.9 (128). Std. dev. ≤1.5% of the mean in every cell.

**Microbenchmarks.** LB_MODE 2 beats ECMP in every cell, by 26.7% on average, and brings JCT from 1.41–1.77× ideal to 1.14–1.28×. The gain hardly depends on message size but does depend on scale: −28 to −32% at 32 and 64 hosts, −18 to −19% at 128, where ECMP already spreads 32 flows per ToR better. Fig. 1b shows the mechanism: no uplink is ever chosen with more than 4 flows (ECMP: up to 8), mean sharing falls from 2.65 to 2.09, and the slowest flow's FCT halves (52.5 → 25.7 ms).

**Robustness.** LB_MODE 2 completes every scenario and beats ECMP in each (Table 2). After the link failure it re-placed 4 flows within 3 µs, at the failed ToR and at a remote ToR whose route to that pod lost the spine. With mixed link speeds a ToR put 4 flows on its 25G uplink and 1–2 on each 10G

| Scenario (32 hosts, 32 MB, seeds 1–3) | ECMP (ms) | LB_MODE 2 (ms) | Δ |
|---|---|---|---|
| ToR–spine link fails at 20 ms | $147.5 \pm 0.4$ | $115.3 \pm 0.8$ | $-21.8\%$ |
| 2 equal-cost paths | $216.6 \pm 8.5$ | $184.1 \pm 4.7$ | $-15.0\%$ |
| 6 equal-cost paths | $131.6 \pm 0.5$ | $73.7 \pm 0.0$ | $-44.0\%$ |
| One pod with 3 uplinks + 25G links to one spine | $162.4 \pm 2.9$ | $134.0 \pm 1.3$ | $-17.5\%$ |
| Two tenants, $2 \times 16$ hosts (seed 1, slowest job) | 134.0 | 95.3 | $-28.8\%$ |

**Table 2.** Robustness: only the topology, `LINK_DOWN` or workload changes, identically for both configs.

uplink (6.25 vs. 5 Gb/s per flow), so normalising by capacity works as intended. In the 128-host two-job challenge (seed 1), jobs 1 and 2 finish 26% and 16% sooner (478.7 → 354.2 and 656.6 → 553.0 ms).

## DISCUSSION

**Strengths.** The fix targets the measured cause with switch-local state and one O(paths) decision per flow. It never reorders packets and needs no host, CC or ECN changes.

**Limitations.** (i) *Local view:* a ToR cannot see a remote spine downlink congested by other ToRs' flows. That is harmless for rings, where each pod receives from one pod, but not in general; CONGA-style feedback Alizadeh et al. (2014) would close the gap. (ii) *Flow count as load proxy:* a short or rate-limited flow counts like an elephant until it ages out. (iii) *Idle timeout:* a flow paused (e.g. by PFC) for more than 500 µs could be re-placed with packets still queued; we did not test PFC storms. (iv) *Diminishing returns:* with many flows per ToR, ECMP already balances well; the rest of the gap is DCQCN and queueing.

## ACKNOWLEDGMENTS

Code, scripts and parts of this report were developed with the assistance of Claude Code (Anthropic); all of it was reviewed and verified by Group 9. Post-processing is in `results/ms2_analyze.py` and `results/ms2_plots.py`.

## REFERENCES

Al-Fares, M., Radhakrishnan, S., Raghavan, B., Huang, N., and Vahdat, A. (2010). Hedera: Dynamic flow scheduling for data center networks. In *Proc. USENIX NSDI*.

Alizadeh, M., Edsall, T., Dharmapurikar, S., Vaidyanathan, R., Chu, K., Fingerhut, A., Lam, V. T., Matus, F., Pan, R., Yadav, N., and Varghese, G. (2014). CONGA: Distributed congestion-aware load balancing for datacenters. In *Proc. ACM SIGCOMM*.

Gangidi, A., Miao, R., Zheng, S., Bondu, S. J., Goes, G., Morsy, H., Puri, R., Riftadi, M., Shetty, A. J., Yang, J., Zhang, S., Fernandez, M. J., Gandham, S., and Zeng, H. (2024). RDMA over Ethernet for distributed AI training at Meta scale. In *Proc. ACM SIGCOMM*.

Ghorbani, S., Yang, Z., Godfrey, P. B., Ganjali, Y., and Firoozshahian, A. (2017). DRILL: Micro load balancing for low-latency data center networks. In *Proc. ACM SIGCOMM*.

Hopps, C. E. (2000). Analysis of an equal-cost multi-path algorithm. RFC 2992.

Raab, M. and Steger, A. (1998). "balls into bins" – a simple and tight analysis. In *Proc. RANDOM*.

Vanini, E., Pan, R., Alizadeh, M., Taheri, P., and Edsall, T. (2017). Let it flow: Resilient asymmetric load balancing with flowlet switching. In *Proc. USENIX NSDI*.

Won, W., Heo, T., Rashidi, S., Sridharan, S., Srinivasan, S., and Krishna, T. (2023). ASTRA-sim2.0: Modeling hierarchical networks and disaggregated systems for large-model training at scale. In *Proc. IEEE ISPASS*.

Zhu, Y., Eran, H., Firestone, D., Guo, C., Lipshteyn, M., Liron, Y., Padhye, J., Raindel, S., Yahia, M. H., and Zhang, M. (2015). Congestion control for large-scale RDMA deployments. In *Proc. ACM SIGCOMM*.